

Codex 使用最佳实践：别把它当聊天机器人，要当成一个可配置的工程队友

原创 AI 技术小林 AI技术小林 2026年4月27日 09:00 广东



一句话定位：**Codex 的关键不是“写一句 prompt 让它生成代码”，而是把它当成一个可以被配置、被训练工作方式、能持续沉淀经验的工程队友。**

这篇文章参考了 OpenAI 官方的 Codex best practices，但不会逐条翻译官方文档，而是整理成更适合日常使用的实践方法。

先说结论

很多人刚开始用 Codex，会把它当成一个更强的代码聊天机器人：

- 描述一个需求
- 等它生成代码
- 看看能不能跑
- 不行再继续追问

这种方式能用，但很容易不稳定。

更好的方式是：**把 Codex 放进一套工程 workflow 里。**

也就是说，不只是让它“写代码”，还要让它知道：

- 目标是什么
- 项目上下文在哪里
- 需要遵守哪些约束
- 怎么判断任务完成
- 应该跑哪些测试
- 哪些经验以后要沉淀下来

当这些东西逐渐固定下来，Codex 的表现会明显更像一个熟悉项目的队友，而不是每次都从零开始的临时助手。

1. Prompt 不用花哨，但上下文要完整

Codex 已经足够强，即使 prompt 写得不完美，也经常能给出不错的结果。

但在真实项目里，尤其是代码量较大、约束较多的项目里，真正影响结果的不是文案写得多漂亮，而是你有没有把关键上下文说清楚。

一个比较稳的任务描述，可以包含四块：

1. **目标**：你到底想改什么、做什么。
2. **上下文**：相关文件、目录、错误日志、接口文档、历史实现在哪里。
3. **约束**：代码风格、架构边界、安全要求、不能改的东西。
4. **完成标准**：什么情况算完成，比如测试通过、bug 不再复现、页面行为符合预期。

比如不要只说：



帮我修一下登录问题。

可以改成：



目标：修复用户登录后偶尔跳回首页的问题。

上下文：登录逻辑在 `src/auth`，路由守卫在 `src/router`，最近错误日志见 `logs/login-error.log`

约束：不要改数据库结构，不要重写登录流程，只修复当前跳转问题。

完成标准：补充或更新测试，确认登录后能回到原访问页面，相关测试通过。

这不是为了“教模型怎么写代码”，而是为了减少它做无谓假设。

2. 复杂任务先 Plan，再让它动手

如果任务简单，比如改一个文案、修一个小 bug，可以直接让 Codex 做。

但如果任务具备这些特征，就应该先让它计划：

- 涉及多个模块
- 需求还不清楚
- 可能影响现有架构
- 需要排查原因
- 需要分阶段上线

这时不要急着说“直接改”，而是先让它：

- 阅读相关代码
- 复述它对问题的理解
- 找出可能的风险点
- 给出修改方案
- 说明验证方式

官方文档里也强调了 Plan mode。它的价值不是让流程变慢，而是先把“怎么做”暴露出来，避免 Codex 一上来就改一大片代码。

一个好用的说法是：



先不要写代码。请先阅读相关文件，给出你的理解、修改计划、风险点和验证方式。等我确认后再开始实现。

对于不确定的需求，还可以让 Codex 反过来采访你：



我现在只有一个粗略想法，请你先问我问题，帮我把需求变成可执行的开发任务。

这一步对产品功能、重构、复杂 bug 都很有用。

3. 把重复要求写进 AGENTS.md

如果你发现自己经常反复提醒 Codex：

- 不要改无关文件
- 改完要跑测试
- 提交前先看 diff
- API 返回格式不能变
- 某些目录不能碰
- 这个项目用 pnpm，不要用 npm

那这些就不应该每次都写进 prompt，而应该沉淀到 **AGENTS.md**。

可以把 **AGENTS.md** 理解成“给 AI Agent 看的项目 README”。

它适合写：

- 项目目录结构
- 本地启动方式
- 测试、构建、lint 命令
- 代码风格和工程约定
- Pull Request 要求
- 禁止事项
- 什么情况算完成

官方建议可以用 `/init` 快速生成一个初版，但不要停在生成结果上。真正有价值的是根据项目实际情况持续修改。

一个短但准确的 **AGENTS.md**，通常比一份很长但空泛的规则更有用。

我更建议这样维护：

1. 先写最核心的启动、测试、约束。
2. 当 Codex 犯同一个错误第二次，就把规则补进去。
3. 如果文件变得太长，把 code review、发布流程、调试流程拆成独立文档，再从 **AGENTS.md** 引用。

也就是说，**AGENTS.md** 不应该是一次性写完的文档，而应该是团队和 Codex 磨合出来的工作说明书。

4. 配置比临场提醒更稳定

很多 Codex 使用问题，本质不是模型能力问题，而是环境配置问题。

比如：

- 工作目录不对
- 没有写权限
- 默认模型不合适
- sandbox 权限太松或太紧
- 缺少必要工具
- MCP 没配好
- 每次都用命令行参数临时覆盖

官方文档里提到，可以通过 `config.toml` 固化默认配置，包括模型、reasoning effort、sandbox、approval policy、profiles、MCP 等。

一个比较稳的习惯是：

- 个人默认配置放在 `~/.codex/config.toml`
- 项目级配置放在 `.codex/config.toml`
- 临时命令行参数只用于一次性场景

如果是新手，不建议一开始就给 Codex 很大权限。

更安全的方式是：先用默认权限，让它在需要执行高风险命令时请求确认；等你熟悉某个项目和工作流之后，再逐步放宽。

这点很重要：**权限不是越大越好，合适才好。**

5. 不要只让 Codex 写代码，也要让它验证代码

很多人用 Codex 的流程停在“代码写出来了”。

但在工程里，真正的完成应该是：

- 测试补了没有

- 相关测试跑了没有
- lint / format / type check 过了没有
- diff 有没有异常
- 有没有引入回归
- 最终行为是否符合需求

所以你可以直接把验证要求写进任务：



Codex 不是只能生成代码。用得好时，它也可以帮你做：

- 补测试
- 跑测试
- 看日志
- review diff
- 找潜在回归
- 根据 checklist 做 PR review

官方也提到 `/review` 这类能力。对团队来说，如果能把 code review 规则写成 `code_review.md`，再从 `AGENTS.md` 引用，就可以让 Codex 的 review 行为更稳定。

6. 外部上下文用 MCP，不要一直复制粘贴

有些上下文不在代码仓库里，比如：

- issue 系统
- 文档系统
- 数据库
- 监控平台
- 日志系统
- 内部 API
- 设计稿

如果每次都手动复制粘贴，很快就会变成负担，也容易过期。

这时可以考虑 MCP。

MCP 的价值不是“把所有工具都接进去”，而是让 Codex 能稳定访问某些真正高频、会变化、和任务强相关的信息。

比较适合接 MCP 的场景：

- 经常需要查 issue / PR / CI 状态
- 经常需要读内部文档
- 经常需要查日志或监控
- 多人协作时希望上下文来源一致

但不要一上来就把所有工具都接进去。更好的方式是：先找一个你每天都在手动复制的信息源，把它接好，用顺了再扩展。

7. 重复工作做成 Skill

如果一个流程你已经重复做了很多次，而且每次都要写一大段 prompt，那它就应该变成 Skill。

Skill 适合封装一类稳定任务，比如：

- 日志排查
- 发布说明生成
- PR checklist review
- 数据迁移计划
- 事故复盘摘要
- 标准调试流程

一个好的 Skill 不需要一开始覆盖所有边界。

更现实的做法是：

1. 先选一个典型任务。
2. 把输入、输出、执行步骤写清楚。
3. 跑几次，看哪里容易失败。

4. 再把失败经验补进去。

官方文档里有一句很实用的判断标准：如果你反复使用同一段 prompt，或者反复纠正同一个流程，它大概率应该变成一个 Skill。

这也是 Codex 从“会帮你写代码”变成“懂你工作方式”的关键。

8. 稳定流程再自动化

自动化很诱人，但不要太早。

如果一个任务还需要你频繁盯着、反复纠正，就不适合直接自动化。

更好的顺序是：

1. 先手动跑通。
2. 把方法沉淀成 Skill。
3. 确认输出稳定。
4. 再放到 automation 里定时执行。

适合自动化的任务包括：

- 汇总最近 commits
- 扫描潜在 bug
- 生成 release notes
- 检查 CI 失败
- 生成站会摘要
- 定期跑某类分析报告

可以简单理解为：**Skill 定义方法，Automation 定义时间。**

如果方法还不稳定，先别急着定时跑。

9. 长任务要管理 session，不要一个线程聊到底

Codex 的 session 不只是聊天记录，它会积累上下文、决策和中间状态。

所以 session 管理会直接影响结果质量。

一个实用原则是：一个 session 对应一个相对完整的任务。

不要一个项目永远用同一个 session。这样上下文会越来越臃肿，后面反而更容易跑偏。

如果任务还在同一个问题里，继续用同一个线程是合理的，因为它保留了推理过程。

但如果任务已经分叉，就应该 fork 新线程，或者用 subagent 去做边界清晰的子任务，比如：

- 代码探索
- 测试补充
- 日志分析
- 方案对比
- 风险排查

主线程负责推进核心判断，子线程负责消化局部信息。这样更容易保持清晰。

常见误区

最后总结几个常见误区：

- 把长期规则全塞进 prompt，而不是放进 AGENTS.md 或 Skill。
- 只让 Codex 写代码，不告诉它怎么运行、测试、验证。
- 复杂任务不先计划，直接让它改。
- 一开始就给它过大的本机权限。
- 多个 live thread 同时改同一批文件，却不用 git worktree 隔离。
- 还没稳定的流程，过早做 automation。
- 把 Codex 当成必须一步步盯着的工具，而不是可以并行工作的队友。
- 一个项目只开一个巨大 session，长期混用所有任务。

结论

Codex 的最佳实践，其实不是某个神奇 prompt。

真正重要的是把它纳入工程化使用方式：

- 用清晰上下文启动任务
- 复杂需求先计划
- 用 **AGENTS.md** 沉淀项目规则
- 用配置保证一致性
- 用测试和 review 闭环
- 用 MCP 接入外部上下文
- 用 Skill 固化重复流程
- 用 automation 放大稳定工作流
- 用 session 管理保持上下文干净

如果只把 Codex 当成“代码生成器”，它能帮你省一些时间。

但如果把它当成一个可配置、可复盘、可沉淀经验的工程队友，它带来的效率提升会大很多。

END



AI技术小林

分享 AI、技术、闲聊

23篇原创内容

公众号

持续分享 **AI**、技术、工具和实战干货，关注前沿趋势，也关注真实落地。

如果你想获取这些内容：

- 1 AI 实用技巧
- 2 技术经验总结
- 3 工具与效率方法
- 4 实战案例拆解

欢迎关注公众号「**AI技术小林**」，第一时间获取更多高质量内容。

如果想加入微信群，和更多朋友一起交流学习，后台回复「[加群](#)」即可。

喜欢作者

“ 支持他一下 ”

阅读原文